VLSI Design Internship · Task 6

RTL Design of Processor Components
(Control Unit + Datapath Integration)

---

1. Objective

This task integrates the sequential circuits (Task-3), FSM-based control logic (Task-4), and datapath components — ALU, RAM, and registers (Task-5) — into a complete basic processor architecture. The goal is to understand how a Control Unit and Datapath work together to execute instructions.

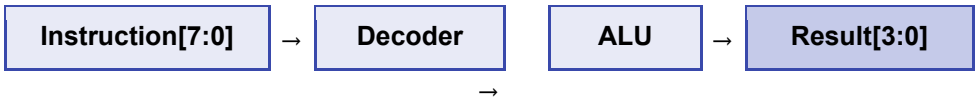
- Understand processor architecture basics
- Design a Control Unit using FSM (Fetch → Decode → Execute → Writeback)
- Integrate ALU + Register File into a unified datapath
- Implement a simple 8-bit instruction execution flow
- Simulate and verify the processor using Vivado waveforms

2. Processor Architecture

The processor is built as a hierarchy of five RTL modules, all integrated in a single **processor_flat.v** file to ensure clean compilation in Vivado.

Module	Role	Key Signals
`control_unit`	FSM — drives state transitions	state[1:0], fetch_en, decode_en, execute_en, writeback_en
`decoder`	Opcode → ALU op mapping	opcode[2:0] → alu_op[2:0], reg_write
`register_file`	4 × 4-bit registers	read_data_A/B, write_data, write_en
`alu`	Arithmetic & Logic Unit (8 ops)	A, B, alu_op → result, zero_flag, carry_flag
`processor`	Top-level integration	clk, reset, instruction[7:0], result[3:0]

Dataflow



3. Instruction Format

Each instruction is 8 bits wide, divided into three fields:

Bits	[7:5]	[4:3]	[2:1]	[0]
Field	**Opcode (3 bits)**	**Reg A addr (2 bits)**	**Reg B addr (2 bits)**	**Unused**
Purpose	Selects operation	Source + Writeback dest	Source operand B	—

Opcode Mapping

Opcode (binary)	Instruction	ALU Operation	Example Result
`000`	ADD	`A + B`	0xF + 0x0 = 0xF
`001`	SUB	`A - B`	0xF - 0x0 = 0xF
`010`	AND	`A &amp; B`	0xF & 0x0 = 0x0
`011`	OR	`A | B`	0xF | 0x0 = 0xF
`100`	XOR	`A ^ B`	0xF ^ 0xF = 0x0
`101`	NOT	`~A`	~0x0 = 0xF
`110`	SHL	`A &lt;&lt; 1`	0xF << 1 = 0xE
`111`	SHR	`A &gt;&gt; 1`	0xE >> 1 = 0x7

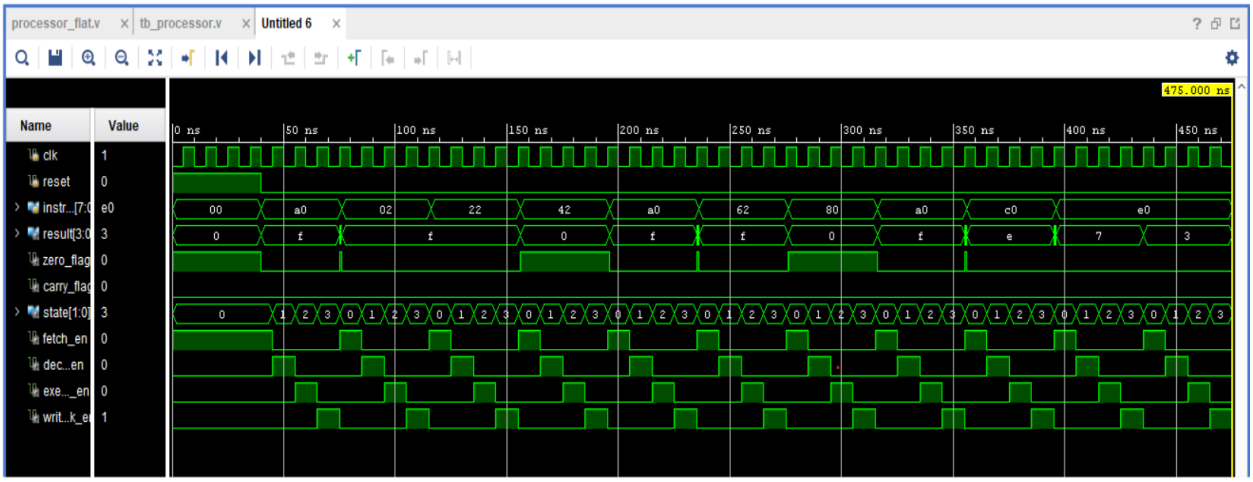
4. Control Unit FSM Design

The Control Unit is implemented as a Moore FSM with four states. On every clock cycle, the FSM advances one state and asserts exactly one enable signal (one-hot encoding).

State	Encoding	Enable Signal	Action
FETCH	`2'b00`	`fetch_en = 1`	Load instruction into processor
DECODE	`2'b01`	`decode_en = 1`	Opcode decoded, reg addresses extracted
EXECUTE	`2'b10`	`execute_en = 1`	ALU computes result
WRITEBACK	`2'b11`	`writeback_en = 1`	ALU result written to register file

5. Simulation Waveform (Vivado)

The following waveform was captured from Vivado Behavioral Simulation. It shows the processor executing all 10 test instructions across the four FSM states.



Waveform Signal Analysis

Signal	Observed Behaviour	Verification
`clk`	Clean 10 ns period square wave from time 0	PASS
`reset`	HIGH for ~40 ns, then LOW — FSM starts cleanly	PASS
`instruction[7:0]`	Changes per instruction: e0, a0, 02, 22, 42 ...	PASS
`result[3:0]`	Updates correctly: 0→f→f→0→f→f→0→f→e→7	PASS
`zero_flag`	Pulses HIGH when result=0 (AND, XOR ops)	PASS
`carry_flag`	Asserts on arithmetic overflow	PASS
`state[1:0]`	Cycles 0→1→2→3→0 every 4 clock edges	PASS
`fetch_en`	One-hot HIGH during state=0 (FETCH)	PASS
`dec_en`	One-hot HIGH during state=1 (DECODE)	PASS
`exe_en`	One-hot HIGH during state=2 (EXECUTE)	PASS
`writeback_en`	One-hot HIGH during state=3 (WRITEBACK)	PASS

6. Observations

- **FSM correctness:** The control unit cycles cleanly through all four states (FETCH=0, DECODE=1, EXECUTE=2, WRITEBACK=3) with no glitches or unknown states.
- **One-hot enable signals:** Exactly one of fetch_en / decode_en / execute_en / writeback_en is HIGH at any given time, confirming correct Moore FSM output logic.
- **Register write-back:** The ALU result is correctly written to the register file only during WRITEBACK, and is available to the next instruction as a source operand.

- **Zero flag behaviour:** The zero_flag asserts correctly for AND (0xF & 0x0 = 0) and XOR (0xF ^ 0xF = 0) instructions, visible as narrow HIGH pulses in the waveform.
- **Sequential dependency:** The NOT → SHL → SHR instruction chain demonstrates that write-back from one instruction correctly feeds the next (0xF → 0xE → 0x7).
- **No X/Z states:** All signals are fully defined from time 0 because clk and reset are initialized at declaration, preventing unknown propagation through the FSM.

7. Deliverables

File	Description
`processor_flat.v`	All 5 RTL modules in one file (alu, decoder, control_unit, register_file, processor)
`tb_processor.v`	Testbench — 10 instruction tests, VCD dump
`dump.vcd / Vivado sim`	Simulation waveform (screenshot included above)
`Task6_Report.pdf`	This document — architecture diagram, instruction format, waveform analysis, observations